COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Jul 16, 2026	**DATE SUBMITTED:**	Jul 21, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

What procedures did you perform?

OUTPUT

Screenshot

table2.1_reyes.cpp

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

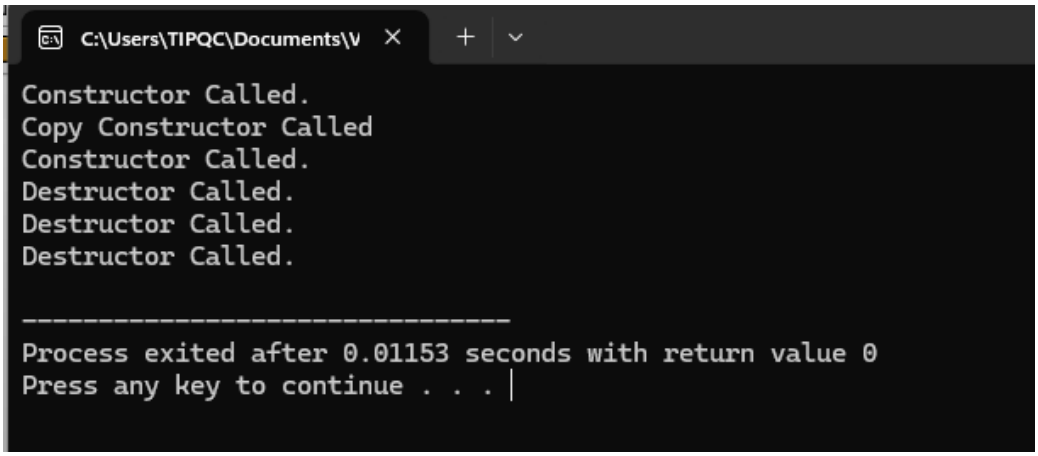
	
Observation	The code first created two libraries which are the iostream and string.h, this will enable the code to work and activate the manipulation of C++ strings. After creating those, it creates the student Class with its private attributes studentName and studentAge. Then, from line 9 to 14, it defines the constructor, and eventually in the following lines, the big three which are the deconstructor, copy constructor, as well as its display attributes are then defined. Doing all those, we come up now to the main file, in which it just defines the student's status. Basically, the main highlight of this code is the creation of the constructor because it's just copy pasting the three students from the given. Then after running, we come up to the three destructors to clear up space.

Table 2-1. Initial Driver Program

Screenshot

table2.1_reyes.cpp

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main(){
    const size_t j = 5;

    Student studentList[j]={};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

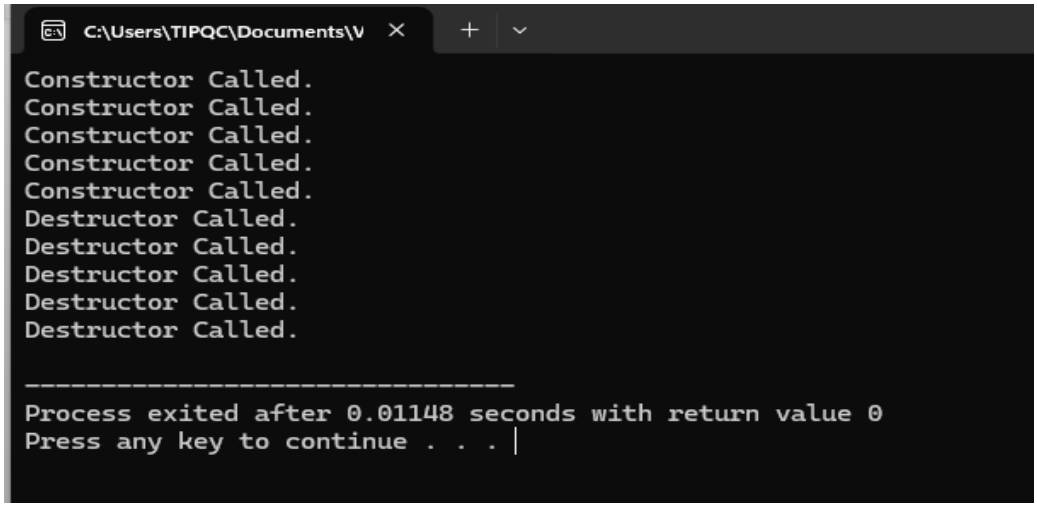
	
Observation	The code here just first initializes and creates room for 5 students. The code then automatically creates 5 “John Doe”, as we have seen from the output that it created 5 constructors. Then, it made a nameList for 5 students which are not connected to the 5 students. Once the program hits the return 5 students are being deleted from the system, as we will see that it has printed 5 destructors calls.

Table 2-2. Modified Driver Program with Student Lists

Loop A

[*] table2.1_reyes.cpp

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "Fenzie Aaron Reyes", int newAge = 19) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main(){
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for (int i = 0; i < j; i++) { // Loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }
    return 0;
}
```

```
C:\Users\TIPQC\Documents\V
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.

--------------------------------
Process exited after 0.009122 seconds with return value 0
```

Observation

The main highlight of this code is that the data from new student in my list are being transported to the studentList[i] = *ptr. The reason why there are 10 constructors being called is that the first five “string newName = Fenzie Aaron Reyes” are being copied by the system into its memory. Then after doing so, the code uses new to build new student real names, so it adds up to another 5 constructors being called. Then after it hits the return function, it eventually deletes the memory.

Loop B

table2.1_reyes.cpp

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "Fenzie Aaron Reyes", int newAge = 19) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    // Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main(){
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

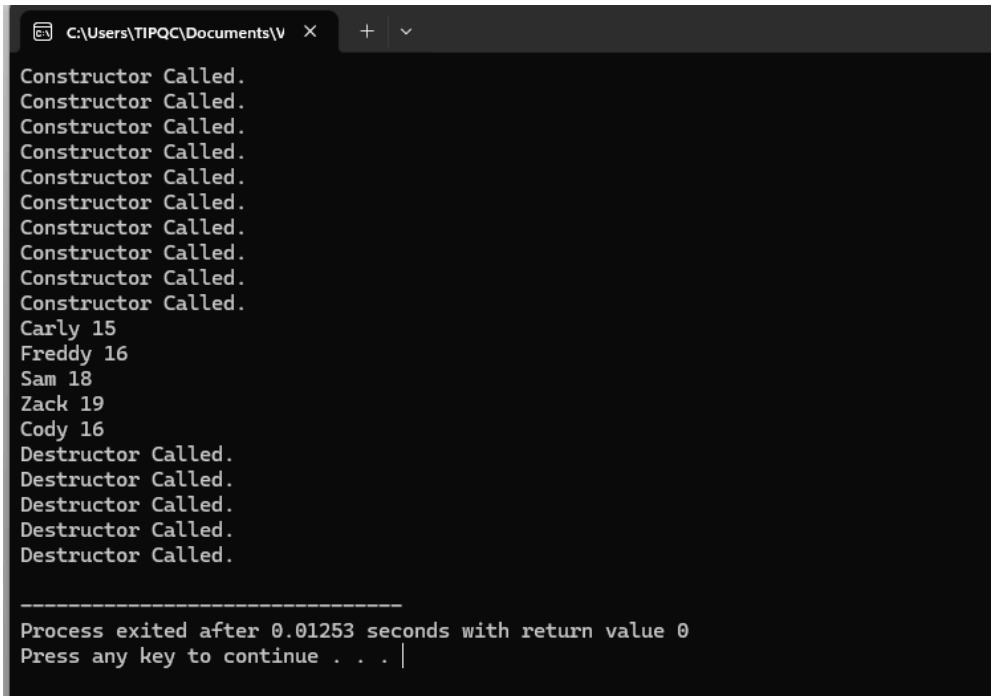
    for (int i = 0; i < j; i++) { // Loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

    for (int i = 0; i < j; i++) { // Loop B
        studentList[i].printDetails();
    }

    return 0;
}
```

```
C:\Users\TIPQC\Documents\V
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.

--------------------------------
Process exited after 0.01253 seconds with return value 0
Press any key to continue . . .
```

Observation	Similarly, in loop B, the 10 constructors are being called by the system with 5 default names of “Fenzie Aaron Reyes” and 5 corrected new names of the students. With the use of for loops from line 44 to 46, it prints the student’s details such as their names and age. Again, as it runs and goes through the return function, it destroys the constructors that have been created.
Output	
Observation	Given all of the activities and procedures that were given, it mainly emphasizes the importance of using constructors in creating a dynamic memory allocation. Having an understanding about the use of constructors given the activity will avoid overcopying and duplicating the same names repeatedly. Moreover, it will also add up to our understanding on how objects are being created and how it eventually destroys as the program executes. In addition to that, I also learned what happens when using “new” to my code and slightly a bit understanding of how my program will avoid crashes. At last, it helps me proofread my code by understanding how data transfer works by differentiating the difference between creating and assigning tasks to a given variable.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

[*] JennasGroceryList.cpp

```cpp
#include <iostream>
#include <string>

//=====================================================
// Base Class: GroceryItem
//=====================================================
class GroceryItem{
protected:
    std::string *name;
    double price;
    int quantity;

public:
    // Constructor
    GroceryItem(std::string n = "", double p = 0.0, int q = 0){
        name = new std::string(n);
        price = p;
        quantity = q;
    }

    // Destructor
    virtual ~GroceryItem(){
        delete name;
    }

    // Copy Constructor
    GroceryItem(const GroceryItem &other){
        name = new std::string(*other.name);
        price = other.price;
        quantity = other.quantity;
    }

    // Copy Assignment Operator
    GroceryItem& operator=(const GroceryItem &other){
        if (this != &other){
            delete name;
            name = new std::string(*other.name);
            price = other.price;
            quantity = other.quantity;
        }

        return *this;
    }

    // Calculate subtotal
    double calculateSum() const{
        return price * quantity;
    }

    double getSum() const{
        return calculateSum();
    }

```

```cpp
    std::string getName() const{
        return *name;
    }

    // Pure Virtual Functions
    virtual void display() const = 0;
    virtual std::string getType() const = 0;
};

//=======================================================
// Fruit Class
//=======================================================
class Fruit : public GroceryItem{
public:
    // Constructor
    Fruit(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q){
    }

    // Copy Constructor
    Fruit(const Fruit &other)
        : GroceryItem(other){
    }

    // Copy Assignment Operator
    Fruit& operator=(const Fruit &other){
        if (this != &other){
            GroceryItem::operator=(other);
        }

        return *this;
    }

    void display() const{
        std::cout << "Type      : Fruit" << std::endl;
        std::cout << "Name      : " << *name << std::endl;
        std::cout << "Price     : PHP " << price << std::endl;
        std::cout << "Quantity  : " << quantity << std::endl;
        std::cout << "Subtotal  : PHP " << calculateSum() << std::endl;
        std::cout << "----------------------------------------"
                  << std::endl;
    }

    std::string getType() const{
        return "Fruit";
    }
};
```

```cpp
//=======================================================
// Vegetable Class
//=======================================================
class Vegetable : public GroceryItem{
public:
    // Constructor
    Vegetable(std::string n = "", double p = 0.0, int q = 0)
        : GroceryItem(n, p, q){
    }

    // Copy Constructor
    Vegetable(const Vegetable &other)
        : GroceryItem(other){
    }

    // Copy Assignment Operator
    Vegetable& operator=(const Vegetable &other){
        if (this != &other)
        {
            GroceryItem::operator=(other);
        }

        return *this;
    }

    void display() const{
        std::cout << "Type      : Vegetable" << std::endl;
        std::cout << "Name      : " << *name << std::endl;
        std::cout << "Price     : PHP " << price << std::endl;
        std::cout << "Quantity  : " << quantity << std::endl;
        std::cout << "Subtotal  : PHP " << calculateSum() << std::endl;
        std::cout << "----------------------------------------"
                  << std::endl;
    }

    std::string getType() const{
        return "Vegetable";
    }
};

//=======================================================
// TotalSum Function
//=======================================================
double TotalSum(GroceryItem *GroceryList[], int size){
    double total = 0.0;

    for (int i = 0; i < size; i++){
        if (GroceryList[i] != NULL){
            total += GroceryList[i]->getSum();
        }
    }
```

```cpp
    return total;
}

//=====================================================
// Main Function
//=====================================================
int main(){
    const int SIZE = 4;

    // Dynamic Memory Allocation
    GroceryItem *GroceryList[SIZE];

    GroceryList[0] = new Fruit("Apple", 10, 7);
    GroceryList[1] = new Fruit("Banana", 10, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50, 10);

    std::cout << "=========================================" << std::endl;
    std::cout << "          JENNA'S GROCERY LIST" << std::endl;
    std::cout << "=========================================" << std::endl;

    // Display all grocery items
    for (int i = 0; i < SIZE; i++){
        GroceryList[i]->display();
    }

    // Display Total Amount
    std::cout << "=========================================" << std::endl;
    std::cout << "Total Amount : PHP " << TotalSum(GroceryList, SIZE) << ".00" << std::endl;
    std::cout << "=========================================" << std::endl;

    // Remove Lettuce
    std::cout << "Removing Lettuce from the Grocery List..." << std::endl;

    delete GroceryList[3];
    GroceryList[3] = NULL;

    // Display Updated Grocery List
    std::cout << "=========================================" << std::endl;
    std::cout << "       UPDATED GROCERY LIST" << std::endl;
    std::cout << "=========================================" << std::endl;

    for (int i = 0; i < SIZE; i++){
        if (GroceryList[i] != NULL){
            GroceryList[i]->display();
        }
    }

    // Display Updated Total
    std::cout << "=========================================" << std::endl;
    std::cout << "Updated Total Amount : PHP " << TotalSum(GroceryList, SIZE) << ".00" << std::endl;
    std::cout << "=========================================" << std::endl;
```

```
207     //=======================================================
208     // Free Remaining Dynamically Allocated Memory
209     //=======================================================
210     for (int i = 0; i < SIZE; i++){
211         if (GroceryList[i] != NULL){
212             delete GroceryList[i];
213             GroceryList[i] = NULL;
214         }
215     }
216
217     std::cout << "All remaining grocery items have been deleted." << std::endl;
218     std::cout << "Program terminated successfully." << std::endl;
219
220     return 0;
221 }
222
```

```
C:\Users\Angel Elayza Reyes\(
==========================================
          JENNA'S GROCERY LIST
==========================================
Type        : Fruit
Name        : Apple
Price       : PHP 10
Quantity    : 7
Subtotal    : PHP 70
------------------------------------------
Type        : Fruit
Name        : Banana
Price       : PHP 10
Quantity    : 8
Subtotal    : PHP 80
------------------------------------------
Type        : Vegetable
Name        : Broccoli
Price       : PHP 60
Quantity    : 12
Subtotal    : PHP 720
------------------------------------------
Type        : Vegetable
Name        : Lettuce
Price       : PHP 50
Quantity    : 10
Subtotal    : PHP 500
------------------------------------------
==========================================
Total Amount : PHP 1370.00
==========================================
```



CODE ANALYSIS:

This program illustrates various aspects of Object-Oriented Programming (OOP) using the base class, named "GroceryItem", and the derived classes such as Fruit and Vegetable for different grocery items. This program uses constructors, destructors, copy constructors, and copy assignment operator functions to ensure proper initialization, copying, and releasing of dynamically allocated memory to prevent memory leaks. The calculateSum() function is used for calculating the subtotal of the items and the TotalSum() function for calculating the total sum of the grocery items. In addition to that, this program shows the use of dynamic memory allocation with the help of the new and delete keywords.

The most important lesson from this assignment is that one should not only utilize object oriented programming principles but also effectively manage the computer memory in order to write well structured, modular and efficient applications. The usage of inheritance and polymorphism allows the code to be concise without any repetitions and to process various types of groceries using one array, named GroceryList. In addition to that, the principles of good programming like encapsulation, modularity of functions and proper resource deallocation are also demonstrated.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of errors and break down of how the code works.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI was only used to define what is missing and what can be improved in the code. In addition to that, it helped break down parts into comprehensible approaches in which I can understand more clearly aside from the fact that I'm also using PythonTutor.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.